

PROJECT SDL

“Simulator Smart Playlist Aplikasi Streaming Musik”

KELOMPOK 2

KELAS : CP

Dosen Pengampun:

Dr. Sri Hartati Wijono



DIBUAT OLEH:

Tavonga John Junior Gwenhure 255314082

Gilbert Gaffin Pramudya

Gilberth Untung / 235314107

Arnoldus Yoseph Freibedemenz Lawe

Stefanus Yiko

PROGRAM STUDI INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS SANATA DHARMA

2026

BAB I

PENDAHULUAN

1.1 Deskripsi Kasus

Di era digital saat ini, layanan streaming musik telah menjadi bagian tak terpisahkan dari kehidupan sehari-hari. Platform seperti Spotify, Apple Music, dan YouTube Music melayani ratusan juta pengguna secara bersamaan, dengan setiap pengguna memiliki daftar putar, antrian lagu, dan riwayat pemutaran yang berbeda-beda. Di balik antarmuka yang sederhana tersebut, terdapat struktur data kompleks yang bekerja secara efisien untuk mengatur alur pemutaran lagu.

Proyek ini mengangkat kasus perancangan sistem Music Player berbasis struktur data, yang mensimulasikan fungsionalitas inti dari sebuah pemutar musik modern. Sistem ini dirancang untuk mengelola playlist lagu secara dinamis, mengatur antrian lagu yang akan diputar berikutnya (Up Next), serta mencatat riwayat pemutaran. Pemilihan kasus ini didasari oleh relevansinya dengan konsep-konsep fundamental dalam Algoritma dan Struktur Data, yakni Doubly Linked List, Queue, dan Stack.

Doubly Linked List dipilih sebagai struktur utama playlist karena kemampuannya untuk melakukan traversal dua arah maju ke lagu berikutnya maupun mundur ke lagu sebelumnya yang merupakan kebutuhan esensial dalam sistem pemutar musik. Queue (antrian) diimplementasikan untuk fitur Up Next, di mana lagu yang ditambahkan akan diputar berdasarkan urutan penambahan ((FIFO)) First In, First Out). Sementara itu, Stack (tumpukan) digunakan untuk menyimpan riwayat pemutaran, sehingga pengguna dapat kembali ke lagu yang sebelumnya diputar menggunakan prinsip LIFO (Last In, First Out). Sebagai pelengkap, algoritma Insertion Sort diterapkan untuk mengurutkan koleksi lagu berdasarkan tahun rilis.

1.2 Spesifikasi Kebutuhan Fungsional

Berdasarkan analisis kasus di atas, sistem Music Player ini harus mampu memenuhi kebutuhan fungsional berikut:

a) Manajemen Playlist (Doubly Linked List)

1. Sistem dapat menambahkan lagu baru ke dalam playlist secara dinamis.

2. Sistem dapat menavigasi playlist ke lagu berikutnya (next) secara siklik — ketika lagu terakhir tercapai, sistem kembali ke lagu pertama.
3. Sistem dapat menavigasi playlist ke lagu sebelumnya (previous) secara siklik — ketika lagu pertama tercapai, sistem kembali ke lagu terakhir.
4. Sistem dapat menampilkan seluruh isi playlist beserta penanda lagu yang sedang aktif diputar.
5. Sistem dapat mencari dan memilih lagu berdasarkan judul, nama artis, atau tahun rilis.

b) Antrian Up Next (Queue)

6. Sistem dapat menambahkan lagu ke dalam antrian Up Next berdasarkan judul lagu.
7. Sistem dapat memutar lagu dari antrian Up Next dengan mengambil lagu paling depan (dequeue) sebelum melanjutkan ke urutan playlist biasa.
8. Sistem dapat menampilkan seluruh isi antrian Up Next.

c) Riwayat Pemutaran (Stack)

9. Sistem dapat menyimpan riwayat lagu yang telah diputar secara otomatis setiap kali lagu berpindah.
10. Sistem dapat memutar kembali lagu sebelumnya dengan mengambil data dari puncak stack riwayat.
11. Sistem dapat menampilkan seluruh riwayat pemutaran dari yang paling terakhir hingga yang paling awal.
12. Sistem dapat memperbesar kapasitas stack secara otomatis (dynamic resizing) ketika kapasitas penuh.

d) Pengurutan Koleksi Lagu (Sorting)

13. Sistem dapat mengurutkan dan menampilkan seluruh koleksi lagu berdasarkan tahun rilis menggunakan algoritma Insertion Sort.
14. Sistem dapat mengurutkan ulang playlist utama berdasarkan tahun rilis secara langsung (in-place sort pada Doubly Linked List).

e) Informasi dan Tampilan

15. Sistem dapat menampilkan lagu yang sedang aktif diputar (Now Playing).
16. Sistem dapat memuat data lagu default secara otomatis saat aplikasi pertama kali dijalankan.
17. Sistem menyediakan antarmuka berbasis teks (CLI) yang interaktif dan mudah digunakan.

BAB II

ANALISIS DAN PERANCANGAN SISTEM

2.1 Perancangan Entitas dan Node (Model Data)

Sebelum bicara soal struktur data yang dipakai, ada baiknya kita pahami dulu objek-objek dasar yang menjadi pondasi sistem ini. Secara garis besar, ada dua lapisan model data di sini: satu lapisan yang merepresentasikan objek dunia nyata (dalam hal ini, sebuah lagu), dan satu lapisan lagi yang bertugas sebagai unit penyusun struktur data itu sendiri.

a) Class Song (Entitas Utama)

Class Song adalah representasi paling sederhana dari objek yang kita kelola sebuah lagu. Di dalamnya tersimpan tiga informasi: judul lagu (title), nama artis (artist), dan tahun rilis (releaseYear). Tiga field itu saja sudah cukup untuk mendeskripsikan sebuah lagu di sistem ini.

Yang menarik dari class Song di proyek ini adalah ia tidak hanya berperan sebagai tempat menyimpan data lagu biasa. Song juga sekaligus berfungsi sebagai node dalam Doubly Linked List, karena di dalamnya ada dua field tambahan: next dan prev, yang masing-masing menunjuk ke lagu berikutnya dan lagu sebelumnya dalam playlist. Pola seperti ini lazim disebut sebagai self-referential object.

Kenapa didesain seperti ini? Alasannya cukup pragmatis daripada membuat class Node terpisah yang hanya membungkus Song, lebih efisien kalau Song langsung membawa pointer-nya sendiri. Memori yang dialokasikan jadi lebih sedikit, dan struktur kodenya pun lebih ringkas.

b) Class QueueNode (Node Wrapper untuk Queue)

Berbeda dengan pendekatan pada DoublyLinkedList, implementasi UpNextQueue justru memilih untuk memisahkan antara data dan node-nya. Di sini ada class QueueNode yang bertugas sebagai pembungkus: ia menyimpan referensi ke objek Song lewat field data, dan punya pointer next untuk menunjuk ke node berikutnya dalam antrian.

Pilihan untuk memisahkan QueueNode dari Song ini bukan tanpa alasan. Pendekatan ini mengikuti prinsip desain yang lebih modular kalau suatu hari tipe data dalam antrian perlu diganti atau diperluas, kita tidak perlu menyentuh class Song sama sekali. Cukup ubah QueueNode-nya saja.

c) Class HistoryStack (Array-Based)

Untuk riwayat pemutaran, pendekatannya berbeda lagi. HistoryStack tidak menggunakan node berbasis pointer sama sekali, melainkan memanfaatkan array Song[] sebagai tempat penyimpanan. Posisi elemen teratas stack dilacak lewat sebuah variabel integer bernama top.

Array dipilih di sini karena operasi utama stack push dan pop hanya berurusan dengan elemen paling atas, yang bisa diakses langsung lewat indeks. Tidak perlu traversal, tidak perlu mengikuti pointer satu per satu. Selain itu, array lebih cache-friendly karena data tersimpan secara berurutan di memori. Satu-satunya kelemahan array adalah ukurannya yang tetap, tapi itu diatasi dengan mekanisme resize otomatis: setiap kali kapasitas penuh, array digandakan ukurannya.

2.2 Pemetaan Struktur Data

Setiap struktur data yang digunakan dalam proyek ini dipilih berdasarkan alasan yang spesifik, bukan sekadar karena familiar atau mudah diimplementasikan. Bagian ini menjelaskan bagaimana setiap kebutuhan fungsional dari Bab 1 dipetakan ke struktur data yang paling sesuai secara logika.

Kebutuhan 1 Manajemen Playlist: menggunakan Doubly Linked List.

- **Alasan:** Playlist itu pada dasarnya butuh dua hal bisa maju ke lagu berikutnya, dan bisa mundur ke lagu sebelumnya. Doubly Linked List pas banget untuk kebutuhan ini karena setiap node punya dua pointer (next dan prev), jadi perpindahan ke dua arah bisa dilakukan dalam $O(1)$ tanpa perlu traversal dari awal. Selain itu, linked list sifatnya dinamis menambah lagu baru cukup dengan mengubah pointer di tail, tanpa perlu menggeser atau merealokasi elemen lain seperti yang terjadi pada array. Navigasi siklik (kembali ke awal saat sudah di lagu terakhir, dan sebaliknya) juga mudah ditangani: cukup periksa apakah `current.next` atau `current.prev` bernilai null.

Kebutuhan 2 Antrian Up Next: menggunakan Queue berbasis Linked List.

- **Alasan:** Fitur Up Next punya aturan sederhana: lagu yang pertama ditambahkan harus jadi yang pertama diputar. Aturan ini persis dengan prinsip FIFO (First In, First Out) yang menjadi ciri khas Queue. Dari semua struktur data yang ada, Queue adalah yang paling tepat secara semantik untuk merepresentasikan antrian seperti ini. Implementasinya memilih linked list (bukan array) supaya operasi enqueue di belakang dan dequeue di depan sama-sama berjalan dalam $O(1)$, tanpa perlu menggeser elemen seperti yang terjadi jika menggunakan array biasa.

Kebutuhan 3 Riwayat Pemutaran: menggunakan Stack berbasis Array Dinamis.

- **Alasan:** Riwayat pemutaran punya perilaku yang cukup khas: lagu yang terakhir diputar adalah yang pertama muncul saat pengguna menekan tombol kembali. Perilaku ini identik dengan prinsip LIFO (Last In, First Out) yang menjadi dasar dari Stack. Konsep yang sama juga dipakai pada fitur Undo di hampir semua editor teks dan aplikasi modern. Untuk implementasinya, array dinamis dipilih karena lebih sederhana dan efisien untuk kasus ini push dan pop hanya melibatkan elemen teratas, yang bisa diakses langsung lewat indeks `top`.

Kebutuhan 4 Pengurutan Koleksi Lagu: menggunakan Array + Insertion Sort.

- **Alasan:** Pengurutan data di linked list secara langsung itu tricky, karena linked list tidak mendukung akses acak (random access). Insertion Sort seperti halnya algoritma sorting berbasis perbandingan lainnya butuh bisa mengakses elemen di posisi mana saja dengan cepat. Maka dari itu, sebelum diurutkan, data dari linked list dikonversi dulu ke array lewat metode `toArray()`. Barulah Insertion Sort dijalankan di sana. Kenapa Insertion Sort? Karena algoritma ini sederhana, mudah ditulis manual, in-place (tidak butuh memori ekstra), dan efisien untuk data berjumlah kecil hingga sedang seperti koleksi 30 lagu default yang ada di sistem ini.

2.3 Pemilihan Algoritma Sorting dan Searching

a) Algoritma Sorting Insertion Sort

Dari sekian banyak algoritma sorting yang ada, proyek ini memilih Insertion Sort. Cara kerjanya sebenarnya cukup intuitif: ambil satu elemen dari bagian yang belum terurut, lalu sisipkan ke posisi yang tepat di antara elemen-elemen yang sudah terurut. Mirip seperti cara kita menyusun kartu di tangan saat bermain.

Ada beberapa alasan kenapa Insertion Sort dipilih di sini, bukan algoritma lain yang mungkin lebih cepat di atas kertas:

1. Implementasinya straightforward dan mudah dipahami saat dibaca ulang penting untuk konteks tugas kuliah yang kodenya perlu dijelaskan.
2. In-place, artinya tidak perlu mengalokasikan array tambahan untuk proses pengurutan.
3. Stabil (stable sort) kalau ada dua lagu dengan tahun rilis yang sama, urutan relatifnya tidak akan berubah.
4. Untuk dataset kecil seperti 30 lagu, selisih performa antara $O(n^2)$ dan $O(n \log n)$ tidak berarti apa-apa secara praktis.

5. Best case-nya $O(n)$ kalau data sudah hampir terurut, algoritma ini sangat efisien. Ini relevan kalau pengguna menambahkan lagu baru yang tahun rilisnya tidak terlalu jauh dari koleksi yang sudah ada.

b) Algoritma Searching Linear Search

Untuk fitur pencarian lagu, sistem menggunakan Linear Search penelusuran satu per satu dari head linked list sampai data ditemukan atau seluruh list habis ditelusuri. Mungkin terdengar sederhana, tapi ada alasan logis kenapa ini yang dipilih dan bukan Binary Search yang lebih efisien.

Alasan utamanya:

6. Data di playlist tidak selalu dalam keadaan terurut berdasarkan judul atau artis dan Binary Search hanya bisa bekerja kalau data sudah terurut berdasarkan kunci yang sama dengan yang dicari.
7. Doubly Linked List tidak mendukung random access. Binary Search butuh bisa loncat langsung ke elemen tengah, sesuatu yang tidak bisa dilakukan di linked list tanpa traversal dari awal yang justru membuatnya lebih lambat dari Linear Search.
8. Dengan 30 lagu, perbedaan antara $O(n)$ dan $O(\log n)$ secara nyata hampir tidak terasa.
9. Linear Search juga mendukung partial-match, misalnya mencari 'Horizon' bisa menemukan 'Bring Me the Horizon'. Binary Search tidak bisa melakukan ini.

BAB III

IMPLEMENTASI SISTEM

3.1 Implementasi Struktur Data Utama

Seluruh struktur data diimplementasikan secara manual dari awal (from scratch) tanpa menggunakan library koleksi bawaan Java seperti `java.util.LinkedList`, `java.util.Stack`, atau `java.util.Queue`. Berikut adalah potongan kode inti dari masing-masing implementasi.

a) Doubly Linked List Inisialisasi dan Operasi Penambahan (addSong)

```
1  package com.mycompany.projectsdl;
2
3  public class DoublyLinkedList {
4
5      private Song head;
6      private Song tail;
7      private Song current;
8      private int size;
9
10     // Konstruktor untuk membuat playlist kosong
11     public DoublyLinkedList() {
12         head = null;
13         tail = null;
14         current = null;
15         size = 0;
16     }
17
18     // Menambahkan lagu baru ke akhir playlist
19     public void addSong(Song s) {
20         if (s == null) return;
21
22         if (head == null) {
23             head = s;
24             tail = s;
25             current = s;
26         } else {
27             tail.next = s;
28             s.prev = tail;
29             tail = s;
30         }
31         s.next = null;
32         size++;
33     }
34
35     // Berpindah ke lagu berikutnya, kembali ke awal jika sudah di akhir
36     public Song next() {
37         if (current == null) return null;
38
39         current = (current.next != null) ? current.next : head;
40         return current;
41     }
42 }
```

b) Doubly Linked List — Navigasi (next dan previous)

```

35 Tabnine | Edit | Test | Explain | Document
36 // Berpindah ke lagu berikutnya, kembali ke awal jika sudah di akhir
37 public Song next() {
38     if (current == null) return null;
39
40     current = (current.next != null) ? current.next : head;
41     return current;
42 }
43
44 Tabnine | Edit | Test | Explain | Document
45 // Berpindah ke lagu sebelumnya, kembali ke akhir jika sudah di awal
46 public Song previous() {
47     if (current == null) return null;
48
49     current = (current.prev != null) ? current.prev : tail;
50     return current;
51 }

```

c) Queue (UpNextQueue) — Inisialisasi, Enqueue, dan Dequeue

```

1  package com.mycompany.projectsdl;
2
3  public class UpNextQueue {
4
5      // Node paling depan (tempat penghapusan data)
6      private QueueNode front;
7
8      // Node paling belakang (tempat penambahan data)
9      private QueueNode rear;
10
11     // Jumlah elemen dalam antrian
12     private int size;
13
14     // Konstruktor untuk membuat antrian kosong
15     public UpNextQueue() {
16         front = null;
17         rear = null;
18         size = 0;
19     }
20
21     // Menambahkan lagu ke belakang antrian (enqueue)
22     public void enqueue(Song s) {
23         if (s == null) {
24             return;
25         }
26
27         QueueNode newNode = new QueueNode(s);
28
29         if (rear == null) {
30             // Jika antrian kosong
31             front = newNode;
32             rear = newNode;
33         } else {
34             rear.setNext(newNode);
35             rear = newNode;
36         }
37         size++;
38
39         System.out.printf(format: " [UP NEXT] Ditambahkan: %s\n", s);
40     }
41
42     // Menghapus dan mengambil lagu dari depan antrian (dequeue)
43     public Song dequeue() {
44         if (isEmpty()) {
45             return null;
46         }

```

```

42 Tabnine | Edit | Test | Explain | Document
43 // Menghapus dan mengambil lagu dari depan antrian (dequeue)
44 public Song dequeue() {
45     if (isEmpty()) {
46         return null;
47     }
48     Song removed = front.getData();
49     front = front.getNext();
50
51     if (front == null) {
52         rear = null;
53     }
54
55     size--;
56     return removed;
57 }
58
59 Tabnine | Edit | Test | Explain | Document
60 // Melihat lagu paling depan tanpa menghapusnya
61 public Song peek() {
62     if (isEmpty()) {
63         return null;
64     }
65     return front.getData();
66 }
67
68 Tabnine | Edit | Test | Explain | Document
69 // Menampilkan seluruh isi antrian Up Next
70 public void display() {
71     if (isEmpty()) {
72         System.out.println(x: " (Antrian Up Next kosong)");
73         return;
74     }
75     QueueNode node = front;
76     int idx = 1;
77
78     while (node != null) {
79         System.out.printf(format: " %2d. %s\n", idx++, node.getData());
80         node = node.getNext();
81     }
82 }

```

```

67 // Menampilkan seluruh isi antrian Up Next
68 public void display() {
69     if (isEmpty()) {
70         System.out.println(x: " (Antrian Up Next kosong)");
71         return;
72     }
73
74     QueueNode node = front;
75     int idx = 1;
76
77     while (node != null) {
78         System.out.printf(format: " %2d. %s\n", idx++, node.getData());
79         node = node.getNext();
80     }
81 }
82
83 // Mengecek apakah antrian kosong
84 public boolean isEmpty() {
85     return size == 0;
86 }
87
88 // Mengambil jumlah lagu dalam antrian
89 public int getSize() {
90     return size;
91 }
92 }

```

d) Stack (HistoryStack) — Inisialisasi, Push, Pop, dan Dynamic Resize

```
1 package com.mycompany.projectsdl;
2
3 public class HistoryStack {
4
5     // Array untuk menyimpan riwayat lagu
6     private Song[] stack;
7
8     // Indeks elemen teratas stack
9     private int top;
10
11     // Kapasitas maksimum stack saat ini
12     private int capacity;
13
14     // Kapasitas awal stack
15     private static final int DEFAULT_CAPACITY = 50;
16
17     // Faktor penambahan kapasitas saat resize
18     private static final int RESIZE_FACTOR = 2;
19
20     // Konstruktor untuk membuat stack kosong
21     public HistoryStack() {
22         capacity = DEFAULT_CAPACITY;
23         stack = new Song[capacity];
24         top = -1;
25     }
26 }
```

```

27 // Menambahkan lagu ke puncak stack
28 public void push(Song s) {
29     if (s == null) return;
30
31     if (top == capacity - 1)
32         resize();
33
34     stack[++top] = s;
35 }
36
37 // Menghapus dan mengambil lagu dari puncak stack
38 public Song pop() {
39     if (isEmpty()) return null;
40
41     Song removed = stack[top];
42     stack[top] = null;
43     top--;
44
45     return removed;
46 }
47
48 // Melihat lagu pada puncak stack tanpa menghapusnya
49 public Song peek() {
50     if (isEmpty()) return null;
51
52     return stack[top];
53 }
54
55 // Menambah kapasitas stack ketika penuh
56 private void resize() {
57     int newCapacity = capacity * RESIZE_FACTOR;
58     Song[] newStack = new Song[newCapacity];
59
60     for (int i = 0; i <= top; i++) {
61         newStack[i] = stack[i];
62     }
63
64     stack = newStack;
65     capacity = newCapacity;
66 }
67

```

```

67
68 // Menampilkan seluruh riwayat lagu dari atas ke bawah
69 public void display() {
70     if (isEmpty()) {
71         System.out.println(x: " (Riwayat kosong)");
72         return;
73     }
74
75     System.out.println(x: " [Puncak -> Bawah]");
76
77     for (int i = top; i >= 0; i--) {
78         System.out.printf(format: " %2d. %s\n",
79             (top - i + 1),
80             stack[i]);
81     }
82 }
83
84 Tabnine | Edit | Test | Explain | Document
85 // Mengecek apakah stack kosong
86 public boolean isEmpty() {
87     return top == -1;
88 }
89
90 Tabnine | Edit | Test | Explain | Document
91 // Mengambil jumlah data dalam stack
92 public int getSize() {
93     return top + 1;
94 }
95
96 Tabnine | Edit | Test | Explain | Document
97 // Mengambil kapasitas maksimum stack saat ini
98 public int getCapacity() {
99     return capacity;
100 }

```

3.2 Implementasi Algoritma

a) Insertion Sort Pengurutan berdasarkan Tahun Rilis

Proses pengurutan pada sistem ini melewati dua tahap. Pertama, Doubly Linked List dikonversi ke array melalui metode `toArray()`. Kedua, array tersebut diurutkan dengan `insertionSort()`. Untuk fitur `sortByYear()` yang mengurutkan playlist secara in-place, hasil array yang terurut kemudian digunakan untuk mengganti data pada setiap node linked list melalui metode `replaceNodeData()` yang memperbarui pointer `prev` dan `next` secara cermat.

b) Linear Search Pencarian Lagu berdasarkan Judul

3.3 Analisis Kompleksitas Waktu (Big-O Notation)

Analisis kompleksitas waktu berikut menggambarkan kinerja algoritma berdasarkan jumlah data n (jumlah lagu dalam sistem).

a) Operasi pada Doubly Linked List

Operasi	Best Case	Average Case	Worst Case
addSong (append ke tail)	$O(1)$	$O(1)$	$O(1)$
next / previous (navigasi)	$O(1)$	$O(1)$	$O(1)$
toArray (konversi ke array)	$O(n)$	$O(n)$	$O(n)$
displayAll (tampil semua)	$O(n)$	$O(n)$	$O(n)$
sortByYear (in-place sort)	$O(n)$	$O(n^2)$	$O(n^2)$

Operasi addSong berjalan dalam $O(1)$ karena sistem selalu menyimpan referensi langsung ke tail, sehingga penambahan di akhir tidak memerlukan traversal. Navigasi next/previous juga $O(1)$ karena hanya memperbarui pointer current. Operasi sortByYear memiliki kompleksitas yang bergantung pada Insertion Sort yang digunakan di dalamnya.

b) Operasi pada UpNextQueue

Operasi	Best Case	Average Case	Worst Case
---------	-----------	--------------	------------

enqueue (tambah di rear)	O(1)	O(1)	O(1)
dequeue (hapus dari front)	O(1)	O(1)	O(1)
peek (lihat front)	O(1)	O(1)	O(1)
display (tampil semua)	O(n)	O(n)	O(n)

Seluruh operasi utama Queue berjalan dalam O(1) karena implementasi berbasis linked list menyimpan referensi langsung ke front (untuk dequeue) dan rear (untuk enqueue). Ini merupakan keunggulan utama dibanding implementasi berbasis array, di mana dequeue dari depan membutuhkan pergeseran O(n).

c) Operasi pada HistoryStack

Operasi	Best Case	Average Case	Worst Case
push (tambah ke puncak)	O(1)*	O(1)*	O(n)**
pop (hapus dari puncak)	O(1)	O(1)	O(1)
peek (lihat puncak)	O(1)	O(1)	O(1)
resize (perbesar kapasitas)	-	-	O(n)

Operasi push berjalan dalam O(1) amortized. Meskipun sesekali memicu resize() yang berjalan O(n), frekuensi resize berkurang secara eksponensial seiring penggandaan kapasitas, sehingga rata-rata biaya per operasi push tetap O(1).

d) Algoritma Sorting — Insertion Sort

Operasi	Best Case	Average Case	Worst Case
insertionSort	$O(n)$	$O(n^2)$	$O(n^2)$
getSortedArray	$O(n)$	$O(n^2)$	$O(n^2)$
sortByYear (in-place)	$O(n)$	$O(n^2)$	$O(n^2)$

Best case $O(n)$ terjadi ketika array sudah dalam keadaan terurut — loop while pada Insertion Sort tidak pernah dieksekusi sehingga hanya satu pass diperlukan. Worst case $O(n^2)$ terjadi ketika data dalam urutan terbalik — setiap elemen harus dibandingkan dan digeser melewati semua elemen yang ada di sebelah kirinya.

e) Algoritma Searching — Linear Search

Operasi	Best Case	Average Case	Worst Case
playSongByTitle	$O(1)$	$O(n)$	$O(n)$
playSongByArtist	$O(1)$	$O(n)$	$O(n)$
playSongByYear	$O(1)$	$O(n)$	$O(n)$
addToUpNextByTitle	$O(1)$	$O(n)$	$O(n)$

Best case $O(1)$ terjadi ketika lagu yang dicari adalah node pertama (head). Worst case $O(n)$ terjadi ketika lagu ada di posisi terakhir atau tidak ditemukan sama sekali, mengharuskan traversal seluruh list. Average case $O(n)$ mengasumsikan distribusi seragam posisi elemen yang dicari.

f) Ringkasan Kompleksitas Keseluruhan Sistem

Fitur / Operasi	Struktur Data	Algoritma	Kompleksitas
Tambah lagu ke playlist	Doubly Linked List	Append ke tail	$O(1)$
Navigasi Next / Previous	Doubly Linked List	Pointer traversal	$O(1)$
Cari lagu (judul/artis/tahun)	Doubly Linked List	Linear Search	$O(n)$
Tambah ke Up Next	Queue (Linked List)	Enqueue	$O(1)$
Putar dari Up Next	Queue (Linked List)	Dequeue	$O(1)$
Simpan riwayat	Stack (Array)	Push	$O(1)^*$
Kembali ke lagu sebelumnya	Stack (Array)	Pop	$O(1)$
Urutkan berdasarkan tahun	Array + DLL	Insertion Sort	$O(n^2)$
Tampilkan perpustakaan urut	Array	Insertion Sort	$O(n^2)$

BAB IV

PENGUJIAN DAN ANALISIS HASIL

4.1 Skenario Pengujian

Pengujian dilakukan secara fungsional (functional testing) untuk memverifikasi bahwa setiap fitur program berjalan sesuai dengan spesifikasi kebutuhan yang telah ditetapkan. Setiap skenario uji dirancang berdasarkan alur nyata yang dapat diakses melalui menu interaktif di class Main dan method-method yang tersedia di class MusicPlayer.

Tabel berikut merangkum 16 skenario pengujian yang mencakup seluruh fitur utama program, mulai dari navigasi playlist menggunakan Doubly Linked List, pengelolaan antrian Up Next menggunakan Queue, riwayat pemutaran menggunakan Stack, hingga pengurutan library menggunakan algoritma Insertion Sort.

Keterangan kolom tabel:

ID : Kode unik skenario pengujian (TC = Test Case)
 Skenario : Nama singkat fitur yang diuji
 Deskripsi Uji : Penjelasan teknis dan method yang diuji
 Data Input : Input yang diberikan selama pengujian
 Expected Output : Keluaran yang diharapkan sesuai spesifikasi
 Actual Output : Keluaran nyata saat program dijalankan
 Status : Pass jika Actual = Expected, Fail jika sebaliknya

ID	Skenario Pengujian	Deskripsi Uji	Data Input	Expected Output	Actual Output	Status
TC-01	Memuat lagu default ke playlist	loadDefaultSongs() memuat 30 lagu ke DoublyLinkedList saat program dijalankan	Program dijalankan	Tampil pesan "30 lagu berhasil dimuat" dan lagu pertama aktif sebagai NOW PLAYING	Tampil pesan "30 lagu berhasil dimuat" dan "Blinding Lights - The Weeknd (2019)" aktif	✓ Pass
TC-02	Navigasi lagu berikutnya (Next Song)	playNext() memindahkan current ke node next pada DoublyLinkedList dan menyimpan lagu lama ke HistoryStack	Pilih menu [1] dari keadaan current = "Blinding Lights"	Current berpindah ke "Hotel California", "Blinding Lights" tersimpan di History	Current berpindah ke "Hotel California", History Stack berisi "Blinding Lights" di puncak	✓ Pass
TC-03	Navigasi lagu sebelumnya (Previous Song)	playPrevious() mengambil lagu terakhir dari HistoryStack (pop) dan menjadikannya current	Pilih menu [2] setelah sebelumnya memutar	Lagu sebelumnya dari riwayat diputar kembali, ditampilkan sebagai "<	Lagu terakhir di History Stack berhasil di-pop dan ditampilkan sebagai current song	✓ Pass

			beberapa lagu	PREVIOUS "		
TC -04	Wrap-around Next ke awal playlist	Saat current berada di node tail, next() harus kembali ke head (wrap-around circular)	Tekan Next berulang hingga melewati lagu terakhir "Chop Suey!"	Setelah "Chop Suey!", current kembali ke "Blinding Lights" (head)	Setelah lagu ke-30 "Chop Suey!", next() berhasil wrap ke "Blinding Lights" (head)	✓ Pass
TC -05	Previous ketika riwayat kosong	playPrevious() menampilkan pesan informatif jika HistoryStack kosong	Pilih menu [2] saat program baru dijalankan (History kosong)	Tampil pesan "Tidak ada riwayat pemutaran sebelumnya."	Tampil pesan "Tidak ada riwayat pemutaran sebelumnya." tanpa error	✓ Pass
TC -06	Menambahk an lagu ke Up Next (Enqueue)	addToUpNextByTitle() menemukan lagu di playlist dan menambahkannya ke ekor UpNextQueue	Input judul: "Billie Jean"	Pesan "[UP NEXT] Ditambahka n: Billie Jean - Michael Jackson (1982)" tampil, lagu masuk ke antrian	Pesan konfirmasi tampil dan "Billie Jean" berhasil masuk ke UpNextQueue sebagai elemen pertama	✓ Pass
TC -07	Putar lagu dari Up Next Queue (Dequeue)	Saat Queue tidak kosong, playNext() mengambil lagu dari depan Queue (dequeue) bukan dari playlist	Tambahka n "Billie Jean" dan "Imagine" ke Up Next, lalu tekan Next	"Billie Jean" diputar dengan label "(dari Up Next)", kemudian "Imagine" pada Next berikutnya	"Billie Jean" diputar dari Queue dengan label "dari Up Next", diikuti "Imagine" pada Next ke-2	✓ Pass
TC -08	Up Next Queue habis, kembali ke playlist	Setelah Queue kosong, playNext() kembali menggunakan navigasi normal DoublyLinkedList	Setelah semua lagu Up Next habis,	Navigasi melanjutkan ke node berikutnya di playlist	Setelah Queue kosong, Next melanjutkan navigasi ke node berikutnya	✓ Pass

			tekan Next lagi	secara normal	di DoublyLinkedL ist	
TC -09	Cari dan putar lagu berdasarkan judul	playSongByTitle() mencari node di playlist (case- insensitive) dan langsung menjadikannya current	Input judul: "bohemia n rhapsody" (huruf kecil)	Lagu "Bohemian Rhapsody - Queen (1975)" langsung diputar	"Bohemian Rhapsody" ditemukan dan diputar meski input menggunakan huruf kecil semua	✓ Pass
TC -10	Cari lagu berdasarkan artis (1 hasil)	playSongByArtist() menemukan tepat 1 lagu dari artis yang dicari, langsung diputar tanpa pilihan	Input artis: "Adele"	"Rolling in the Deep - Adele (2010)" langsung diputar tanpa tampil menu pilihan	"Rolling in the Deep" langsung diputar karena hanya ada 1 lagu Adele di playlist	✓ Pass
TC -11	Cari lagu berdasarkan artis (lebih dari 1 hasil)	playSongByArtist() menampilkan daftar pilihan saat ditemukan lebih dari 1 lagu dari artis yang sama	Input artis: "Bring Me the Horizon"	Tampil daftar 3 lagu BMTH dengan nomor pilihan [1][2][3], user dapat memilih salah satu	Daftar 3 lagu BMTH tampil, user memilih [2] "Kingslayer", lagu tersebut berhasil diputar	✓ Pass
TC -12	Cari lagu berdasarkan tahun rilis (lebih dari 1 hasil)	playSongByYear() menampilkan daftar pilihan saat ada beberapa lagu dari tahun yang sama	Input tahun: 2013	Tampil daftar lagu tahun 2013 ("Shadow Moses" dan "Do I Wanna Know?"), user memilih salah satu	Daftar 2 lagu tahun 2013 tampil, user memilih [1] "Shadow Moses", lagu berhasil diputar	✓ Pass
TC -13	Cari lagu dengan judul	playSongByTitle() menampilkan pesan error jika judul tidak ada di playlist	Input judul: "Lagu	Tampil pesan: "Lagu	Pesan error tampil dengan benar tanpa	✓ Pass

	tidak ditemukan		Tidak Ada"	"Lagu Tidak Ada\" tidak ditemukan di playlist."	crash, program tetap berjalan normal	
TC -14	Tampilkan perpustakaan lagu urutan tahun (Insertion Sort)	showLibrary() memanggil SortingUtil.displaySortedLibrary() yang mengurutkan lagu menggunakan Insertion Sort	Pilih menu [6] dari keadaan playlist tidak berurut	30 lagu tampil dari tahun paling lama (1971) hingga terbaru (2023) secara berurutan	Lagu tampil dari "Stairway to Heaven (1971)" hingga "Flowers (2023)", Insertion Sort $O(n^2)$ berjalan	✓ Pass
TC -15	Tampilkan riwayat pemutaran (HistoryStack)	showHistory() menampilkan isi HistoryStack dari puncak ke bawah (urutan LIFO)	Putar 3 lagu berbeda berturut-turut lalu pilih menu [7]	Tampil riwayat dengan lagu terakhir diputar di posisi paling atas (puncak stack)	Riwayat tampil dengan urutan [Puncak -> Bawah], lagu terakhir yang diputar berada di posisi 1	✓ Pass
TC -16	Demo otomatis semua fitur	runAutoDemo() menjalankan skenario lengkap yang mencakup sorting, navigasi, queue, dan history secara otomatis	Pilih menu [8]	Semua fitur berjalan berurutan tanpa error: library sort, next 3x, tambah queue, dequeue via next, previous 2x, tampilkan history dan playlist	Seluruh rangkaian demo berjalan tanpa exception, output setiap tahap tampil sesuai spesifikasi	✓ Pass

4.2 Tangkapan Layar Eksekusi (Screenshots)

Berikut adalah tangkapan layar dari Command Line Interface (CLI) program saat dijalankan. Screenshots ini merepresentasikan hasil eksekusi nyata dari beberapa skenario pengujian yang telah dijelaskan pada tabel di atas. Tampilan CLI program mencerminkan output

yang dihasilkan oleh masing-masing method di class MusicPlayer dan ditampilkan melalui antarmuka menu di class Main.

Screenshot 1 (TC-01 & TC-02)

Tampilan awal program setelah loadDefaultSongs() berhasil memuat 30 lagu, diikuti eksekusi Next Song yang memindahkan current dan menyimpan lagu lama ke History Stack.

```
Memuat perpustakaan lagu...
30 lagu berhasil dimuat.

> NOW PLAYING: "Blinding Lights" - The Weeknd (2019)

+-----+
| SMART PLAYLIST SIMULATOR |
+-----+
| [0] Play Song              |
| [1] Next Song              |
| [2] Previous Song          |
| [3] Tambah Lagu ke Up Next |
| [4] Lihat Antrean Up Next  |
| [5] Tampilkan Main Playlist|
| [6] Perpustakaan (Urut Tahun)|
| [7] Riwayat Pemutaran      |
| [8] Demo Otomatis          |
| [9] Keluar                 |
+-----+
Pilih menu: 1

> NOW PLAYING: "Hotel California" - Eagles (1977)

-----
Tekan Enter untuk melanjutkan...3

+-----+
| SMART PLAYLIST SIMULATOR |
+-----+
| [0] Play Song              |
| [1] Next Song              |
| [2] Previous Song          |
| [3] Tambah Lagu ke Up Next |
| [4] Lihat Antrean Up Next  |
| [5] Tampilkan Main Playlist|
| [6] Perpustakaan (Urut Tahun)|
| [7] Riwayat Pemutaran      |
| [8] Demo Otomatis          |
| [9] Keluar                 |
+-----+
Pilih menu: 2

< PREVIOUS: "Blinding Lights" - The Weeknd (2019)
```

Screenshot 2 (TC-06 & TC-07)

Proses enqueue dua lagu ("Billie Jean" dan "Imagine") ke UpNextQueue, diikuti dua kali penekanan Next yang memutar lagu dari antrian dengan label "(dari Up Next)".

```
| [0] Play Song |
| [1] Next Song |
| [2] Previous Song |
| [3] Tambah Lagu ke Up Next |
| [4] Lihat Antrean Up Next |
| [5] Tampilkan Main Playlist |
| [6] Perpustakaan (Urut Tahun) |
| [7] Riwayat Pemutaran |
| [8] Demo Otomatis |
| [9] Keluar |
+-----+
Pilih menu: 3

Lagu yang tersedia di playlist:

+-----+
| MAIN PLAYLIST |
+-----+
1. "Blinding Lights" - The Weeknd (2019) ? NOW PLAYING
2. "Hotel California" - Eagles (1977)
3. "In the End" - Linkin Park (2000)
4. "Billie Jean" - Michael Jackson (1982)
5. "Flowers" - Miley Cyrus (2023)
6. "Wonderwall" - Oasis (1995)
7. "Imagine" - John Lennon (1971)
8. "Believer" - Imagine Dragons (2017)
9. "Black Hole Sun" - Soundgarden (1994)
10. "Back in Black" - AC/DC (1980)
11. "Happier Than Ever" - Billie Eilish (2021)
12. "Sweet Child O' Mine" - Guns N' Roses (1987)
13. "Drown" - Bring Me the Horizon (2014)
14. "Bohemian Rhapsody" - Queen (1975)
15. "Mr. Brightside" - The Killers (2004)
16. "Fast Car" - Tracy Chapman (1988)
17. "Levitating" - Dua Lipa (2020)
18. "Smells Like Teen Spirit" - Nirvana (1991)
19. "The Pretender" - Foo Fighters (2007)
20. "Shape of You" - Ed Sheeran (2017)
21. "Sultans of Swing" - Dire Straits (1978)
22. "Bring Me to Life" - Evanescence (2003)
23. "Rolling in the Deep" - Adele (2010)
24. "Kingslayer" - Bring Me the Horizon ft. BABYMETAL (2020)
25. "Creep" - Radiohead (1992)
26. "Shadow Moses" - Bring Me the Horizon (2013)
27. "Stairway to Heaven" - Led Zeppelin (1971)
28. "Welcome to the Black Parade" - My Chemical Romance (2006)
29. "Do I Wanna Know?" - Arctic Monkeys (2013)
30. "Chop Suey!" - System of a Down (2001)

Masukkan judul lagu yang ingin ditambahkan ke Up Next: Billie Jean
[UP NEXT] Ditambahkan: "Billie Jean" - Michael Jackson (1982)
-----
```

Screenshot 3 (TC-09 & TC-11)

Pencarian lagu berdasarkan judul ("bohemian rhapsody") yang langsung memutar lagu, dan pencarian berdasarkan artis ("Bring Me the Horizon") yang menampilkan 3 pilihan lagu.

Tekan Enter untuk melanjutkan...3

```
+-----+
|          SMART PLAYLIST SIMULATOR          |
+-----+
| [0] Play Song                               |
| [1] Next Song                               |
| [2] Previous Song                           |
| [3] Tambah Lagu ke Up Next                  |
| [4] Lihat Antrean Up Next                   |
| [5] Tampilkan Main Playlist                 |
| [6] Perpustakaan (Urut Tahun)               |
| [7] Riwayat Pemutaran                       |
| [8] Demo Otomatis                           |
| [9] Keluar                                  |
+-----+
```

Pilih menu: 0

===== PLAY SONG =====

[1] Song
[2] Singer
[3] Year
[0] Kembali

Pilih filter: 1

Masukkan judul lagu: bohemian rhapsody

> NOW PLAYING: "Bohemian Rhapsody" - Queen (1975)

```

+-----+
Pilih menu: 0

===== PLAY SONG =====
[1] Song
[2] Singer
[3] Year
[0] Kembali
Pilih filter: 3
Masukkan tahun rilis:
Input tahun tidak valid.

-----
Tekan Enter untuk melanjutkan...

+-----+
|          SMART PLAYLIST SIMULATOR          |
+-----+
| [0] Play Song                               |
| [1] Next Song                               |
| [2] Previous Song                           |
| [3] Tambah Lagu ke Up Next                   |
| [4] Lihat Antrean Up Next                     |
| [5] Tampilkan Main Playlist                   |
| [6] Perpustakaan (Urut Tahun)                 |
| [7] Riwayat Pemutaran                         |
| [8] Demo Otomatis                             |
| [9] Keluar                                    |
+-----+
Pilih menu: 0

===== PLAY SONG =====
[1] Song
[2] Singer
[3] Year
[0] Kembali
Pilih filter: 2
Masukkan nama artis: Bring Me the Horizon

Ditemukan 3 lagu dari "Bring Me the Horizon":
[1] "Drown" - Bring Me the Horizon (2014)
[2] "Kingslayer" - Bring Me the Horizon ft. BABYMETAL (2020)
[3] "Shadow Moses" - Bring Me the Horizon (2013)
Pilih nomor lagu: 2

> NOW PLAYING: "Kingslayer" - Bring Me the Horizon ft. BABYMETAL (2020)

```

Screenshot 4 (TC-14)

Tampilan perpustakaan lagu yang telah diurutkan berdasarkan tahun rilis menggunakan Insertion Sort, dari "Stairway to Heaven (1971)" hingga "Flowers (2023)".

```

+-----+
|          SMART PLAYLIST SIMULATOR          |
+-----+
| [0] Play Song                               |
| [1] Next Song                               |
| [2] Previous Song                           |
| [3] Tambah Lagu ke Up Next                  |
| [4] Lihat Antrean Up Next                   |
| [5] Tampilkan Main Playlist                 |
| [6] Perpustakaan (Urut Tahun)               |
| [7] Riwayat Pemutaran                       |
| [8] Demo Otomatis                           |
| [9] Keluar                                  |
+-----+
Pilih menu: 6

+-----+
|          PERPUSTAKAAN LAGU (Urut Tahun)      |
+-----+
1. "Imagine" - John Lennon (1971)
2. "Stairway to Heaven" - Led Zeppelin (1971)
3. "Bohemian Rhapsody" - Queen (1975)
4. "Hotel California" - Eagles (1977)
5. "Sultans of Swing" - Dire Straits (1978)
6. "Back in Black" - AC/DC (1980)
7. "Billie Jean" - Michael Jackson (1982)
8. "Sweet Child O' Mine" - Guns N' Roses (1987)
9. "Fast Car" - Tracy Chapman (1988)
10. "Smells Like Teen Spirit" - Nirvana (1991)
11. "Creep" - Radiohead (1992)
12. "Black Hole Sun" - Soundgarden (1994)
13. "Wonderwall" - Oasis (1995)
14. "In the End" - Linkin Park (2000)
15. "Chop Suey!" - System of a Down (2001)
16. "Bring Me to Life" - Evanescence (2003)
17. "Mr. Brightside" - The Killers (2004)
18. "Welcome to the Black Parade" - My Chemical Romance (2006)
19. "The Pretender" - Foo Fighters (2007)
20. "Rolling in the Deep" - Adele (2010)
21. "Shadow Moses" - Bring Me the Horizon (2013)
22. "Do I Wanna Know?" - Arctic Monkeys (2013)
23. "Drown" - Bring Me the Horizon (2014)
24. "Believer" - Imagine Dragons (2017)
25. "Shape of You" - Ed Sheeran (2017)
26. "Blinding Lights" - The Weeknd (2019)
27. "Levitating" - Dua Lipa (2020)
28. "Kingslayer" - Bring Me the Horizon ft. BABYMETAL (2020)
29. "Happier Than Ever" - Billie Eilish (2021)
30. "Flowers" - Miley Cyrus (2023)

Total: 30 lagu | Algoritma: Insertion Sort | Kompleksitas: O(n^2)

```

Screenshot 5 (TC-15 & TC-16)

Tampilan riwayat pemutaran (HistoryStack) dari puncak ke bawah, dan hasil eksekusi Demo Otomatis (menu [8]) yang menjalankan seluruh fitur secara berurutan.

```
+-----+
|          SMART PLAYLIST SIMULATOR          |
+-----+
| [0] Play Song                               |
| [1] Next Song                               |
| [2] Previous Song                           |
| [3] Tambah Lagu ke Up Next                  |
| [4] Lihat Antrean Up Next                   |
| [5] Tampilkan Main Playlist                 |
| [6] Perpustakaan (Urut Tahun)              |
| [7] Riwayat Pemutaran                      |
| [8] Demo Otomatis                          |
| [9] Keluar                                  |
+-----+
```

Pilih menu: 7

```
+-----+
|          RIWAYAT PEMUTARAN (Stack)          |
+-----+
[Puncak -> Bawah]
1. "Bohemian Rhapsody" - Queen (1975)
2. "Blinding Lights" - The Weeknd (2019)
3. "Blinding Lights" - The Weeknd (2019)
```

Main playlist saat ini:

```
+-----+
|          MAIN PLAYLIST          |
+-----+
1. "Blinding Lights" - The Weeknd (2019)
2. "Hotel California" - Eagles (1977)
3. "In the End" - Linkin Park (2000)
4. "Billie Jean" - Michael Jackson (1982)
5. "Flowers" - Miley Cyrus (2023)
6. "Wonderwall" - Oasis (1995)
7. "Imagine" - John Lennon (1971)
8. "Believer" - Imagine Dragons (2017)
9. "Black Hole Sun" - Soundgarden (1994)
10. "Back in Black" - AC/DC (1980)
11. "Happier Than Ever" - Billie Eilish (2021)
12. "Sweet Child O' Mine" - Guns N' Roses (1987)
13. "Drown" - Bring Me the Horizon (2014)
14. "Bohemian Rhapsody" - Queen (1975)
15. "Mr. Brightside" - The Killers (2004)
16. "Fast Car" - Tracy Chapman (1988)
17. "Levitating" - Dua Lipa (2020)
18. "Smells Like Teen Spirit" - Nirvana (1991)
19. "The Pretender" - Foo Fighters (2007)
20. "Shape of You" - Ed Sheeran (2017)
21. "Sultans of Swing" - Dire Straits (1978)
22. "Bring Me to Life" - Evanescence (2003)
23. "Rolling in the Deep" - Adele (2010)
24. "Kingslayer" - Bring Me the Horizon ft. BABYMETAL (2020)
25. "Creep" - Radiohead (1992)
26. "Shadow Moses" - Bring Me the Horizon (2013) ? NOW PLAYING
27. "Stairway to Heaven" - Led Zeppelin (1971)
28. "Welcome to the Black Parade" - My Chemical Romance (2006)
29. "Do I Wanna Know?" - Arctic Monkeys (2013)
30. "Chop Suey!" - System of a Down (2001)
```

```
=====
DEMO SELESAI
=====
```

BAB V

PENUTUP

5.1 Kesimpulan

Proyek ini berhasil merancang dan mengimplementasikan sebuah sistem pemutar musik berbasis konsol yang memanfaatkan tiga struktur data utama, yakni Doubly Linked List, Queue, dan Stack. Ketiga struktur data tersebut tidak sekadar digunakan sebagai formalitas akademis, melainkan dipilih secara sadar karena karakteristiknya yang paling sesuai dengan kebutuhan nyata dari masing-masing fitur yang dibangun.

Doubly Linked List terbukti menjadi pilihan yang tepat untuk menyimpan dan mengelola playlist. Kemampuan traversal dua arah yang dimilikinya memungkinkan navigasi lagu ke depan maupun ke belakang dengan kompleksitas $O(1)$ per langkah, serta mendukung fitur wrap-around yang membuat pengalaman mendengarkan musik terasa mulus dan alami. Tanpa struktur data ini, implementasi navigasi dua arah akan jauh lebih rumit dan tidak efisien.

Queue dipilih untuk fitur Up Next karena sifatnya yang FIFO (First In, First Out) sangat mencerminkan perilaku antrian lagu dalam dunia nyata lagu yang pertama kali ditambahkan ke antrian adalah lagu yang pertama kali diputar. Implementasi berbasis linked node memberikan fleksibilitas ukuran antrian yang tidak dibatasi memori statis.

Stack diterapkan pada fitur riwayat pemutaran karena sifat LIFO-nya (Last In, First Out) secara alami merepresentasikan konsep "putar ulang lagu terakhir". Setiap kali pengguna ingin kembali ke lagu yang baru saja didengar, operasi `pop()` langsung memberikan lagu yang paling relevan tanpa memerlukan pencarian tambahan.

Algoritma Insertion Sort yang diimplementasikan pada `SortingUtil` berhasil mengurutkan playlist berdasarkan tahun rilis dengan kompleksitas waktu $O(n^2)$ pada kasus terburuk. Meskipun bukan algoritma yang paling efisien untuk data besar, Insertion Sort dipilih karena sifatnya yang stabil, mudah diimplementasikan, dan cukup efisien untuk ukuran playlist yang umum digunakan.

Secara keseluruhan, seluruh fitur yang dirancang — yaitu navigasi playlist, antrian Up Next, riwayat pemutaran, dan pengurutan — telah berhasil diimplementasikan dan bekerja secara terintegrasi melalui class `MusicPlayer` dan antarmuka menu interaktif di Main. Studi kasus ini

membuktikan bahwa pemilihan struktur data yang tepat memiliki dampak langsung terhadap kemudahan implementasi, keterbacaan kode, dan efisiensi operasional program.

5.2 Pembagian Tugas Kelompok

Pengerjaan proyek ini dilakukan secara kolaboratif oleh seluruh anggota kelompok. Setiap anggota memiliki tanggung jawab utama atas komponen tertentu, namun tetap berkoordinasi satu sama lain untuk memastikan integrasi antar komponen berjalan dengan baik. Berikut adalah rincian pembagian tugas yang telah disepakati:

Nama	File / Komponen	Tanggung Jawab
Tavonga John Junior Gwenhure	Song.java	Merancang model data lagu (atribut title, artist, releaseYear), konstruktor, getter/setter, dan method toString(). Bertanggung jawab sebagai fondasi node untuk seluruh struktur data.
Stefanus Yiko	QueueNode.java	Merancang node antrian (QueueNode) dengan atribut data (Song) dan pointer next, beserta konstruktor dan accessor method. Berperan sebagai unit penyusun struktur Queue.
Gilbert Gaffin Pramudya	UpNextQueue.java	Implementasi struktur data Queue menggunakan linked node untuk fitur Up Next. Mencakup operasi enqueue, dequeue, peek, isEmpty, dan display. Mengintegrasikan antrian dengan alur pemutaran musik.
Arnoldus Yoseph Freibedemenz Lawe	HistoryStack.java MusicPlayer.java	Implementasi struktur data Stack untuk riwayat lagu yang pernah diputar. Mencakup operasi push, pop, peek, dan display. Sekaligus merancang class MusicPlayer sebagai pengendali utama yang mengintegrasikan semua struktur data.

Gilberth Untung	SortingUtil.java Main.java Diagram UML	Implementasi algoritma Insertion Sort untuk pengurutan playlist berdasarkan tahun rilis, beserta analisis kompleksitas Big-O. Merancang class Main sebagai titik masuk program dan antarmuka menu interaktif. Menyusun laporan, diagram UML, dan melakukan pengujian (testing).
--------------------	--	---

Meskipun setiap anggota memiliki tanggung jawab utama seperti yang tercantum di atas, seluruh proses pengerjaan dilakukan dalam semangat kerja tim. Diskusi kelompok dilakukan secara berkala untuk membahas desain antarmuka antar class, menyelesaikan konflik integrasi, dan memastikan konsistensi penamaan serta gaya penulisan kode di seluruh proyek.

5.3 Riwayat Commit Repositori

Berikut adalah daftar commit yang mencerminkan proses pengembangan proyek secara bertahap. Setiap commit merepresentasikan satu unit perubahan yang bermakna, mulai dari inisialisasi class hingga penyempurnaan fitur dan dokumentasi.

File	Kontributor	Pesan Commit
Song.java	Tavonga	feat: inisialisasi class Song dengan atribut title, artist, releaseYear
Song.java	Tavonga	feat: tambah konstruktor dan getter/setter untuk semua atribut
Song.java	Tavonga	feat: implementasi method toString() dengan format display yang rapi
Song.java	Tavonga	refactor: tambah validasi input pada setter untuk mencegah nilai null
DoublyLinkedList.java	Kelompok	feat: inisialisasi class DoublyLinkedList dengan pointer head, tail, current
DoublyLinkedList.java	Kelompok	feat: implementasi addSong() — penambahan node ke akhir list
DoublyLinkedList.java	Kelompok	feat: implementasi navigasi next() dengan wrap-around ke head

DoublyLinkedList.java	Kelompok	feat: implementasi navigasi previous() dengan wrap-around ke tail
DoublyLinkedList.java	Kelompok	feat: implementasi sortByYear() menggunakan SortingUtil.insertionSort()
DoublyLinkedList.java	Kelompok	feat: implementasi displayAll() dengan penanda NOW PLAYING
QueueNode.java	Evan	feat: inisialisasi class QueueNode dengan atribut data (Song) dan next
QueueNode.java	Evan	feat: tambah konstruktor, getData(), getNext(), dan setNext()
UpNextQueue.java	Gilbert Gaffin	feat: inisialisasi UpNextQueue dengan pointer front, rear, dan size
UpNextQueue.java	Gilbert Gaffin	feat: implementasi enqueue() — menambahkan lagu ke belakang antrian
UpNextQueue.java	Gilbert Gaffin	feat: implementasi dequeue() — mengambil lagu dari depan antrian
UpNextQueue.java	Gilbert Gaffin	feat: implementasi peek() dan isEmpty() sebagai operasi pendukung
UpNextQueue.java	Gilbert Gaffin	feat: implementasi display() untuk menampilkan isi antrian Up Next
HistoryStack.java	Oris	feat: inisialisasi HistoryStack menggunakan array statis dengan pointer top
HistoryStack.java	Oris	feat: implementasi push() — menyimpan lagu ke tumpukan riwayat
HistoryStack.java	Oris	feat: implementasi pop() — mengambil lagu terakhir dari riwayat
HistoryStack.java	Oris	feat: implementasi peek() dan isFull() untuk validasi kapasitas stack
HistoryStack.java	Oris	feat: implementasi displayHistory() untuk menampilkan riwayat putar
SortingUtil.java	Gilberth Untung	feat: implementasi insertionSort() untuk array Song berdasarkan tahun

SortingUtil.java	Gilberth Untung	docs: tambah komentar analisis kompleksitas Big-O $O(n^2)$ pada insertionSort
MusicPlayer.java	Arnoldus Yoseph Freibedemenz Lawe	feat: inisialisasi MusicPlayer — integrasi playlist, queue, dan stack
MusicPlayer.java	Arnoldus Yoseph Freibedemenz Lawe	feat: implementasi playCurrent() — memutar lagu dan mendorong ke history
MusicPlayer.java	Arnoldus Yoseph Freibedemenz Lawe	feat: implementasi playNext() dan playPrevious() dengan navigasi list
MusicPlayer.java	Arnoldus Yoseph Freibedemenz Lawe	feat: implementasi addToQueue() dan playFromQueue() untuk antrian Up Next
MusicPlayer.java	Arnoldus Yoseph Freibedemenz Lawe	feat: implementasi playFromHistory() untuk memutar ulang dari riwayat
Main.java	Gilberth Untung	feat: inisialisasi Main.java dengan data lagu awal dan setup MusicPlayer
Main.java	Gilberth Untung	feat: implementasi menu interaktif dengan Scanner untuk navigasi pengguna
Main.java	Gilberth Untung	feat: integrasi semua fitur (sort, queue, history) ke dalam menu utama
Main.java	Gilberth Untung	docs: tambah diagram UML class diagram sebagai lampiran dokumentasi

LAMPIRAN

• Tautan Repositori Kode Sumber

Platform : GitHub

Repository : <https://github.com/Sanata-Dharma-University/uts2sdl-c-kelompok-2>

Diagram UML

Diagram UML Class Diagram terlampir pada halaman berikutnya sebagai visualisasi hubungan antar class dalam proyek ini. Diagram tersebut menggambarkan atribut, method, dan relasi (asosiasi, dependensi) antar class Song, DoublyLinkedList, QueueNode, UpNextQueue, HistoryStack, SortingUtil, MusicPlayer, dan Main.

